

Documentation technique

Vite & Gourmand — application de restauration événementielle

Carla Ferrero — ECF Développeur Web et Web Mobile — septembre 2026

Réflexions initiales et choix techniques

1. Analyse du besoin

Vite & Gourmand (Julie et José, traiteur à Bordeaux depuis 25 ans) envoie aujourd'hui ses menus par e-mail à ses habitués. L'application doit :

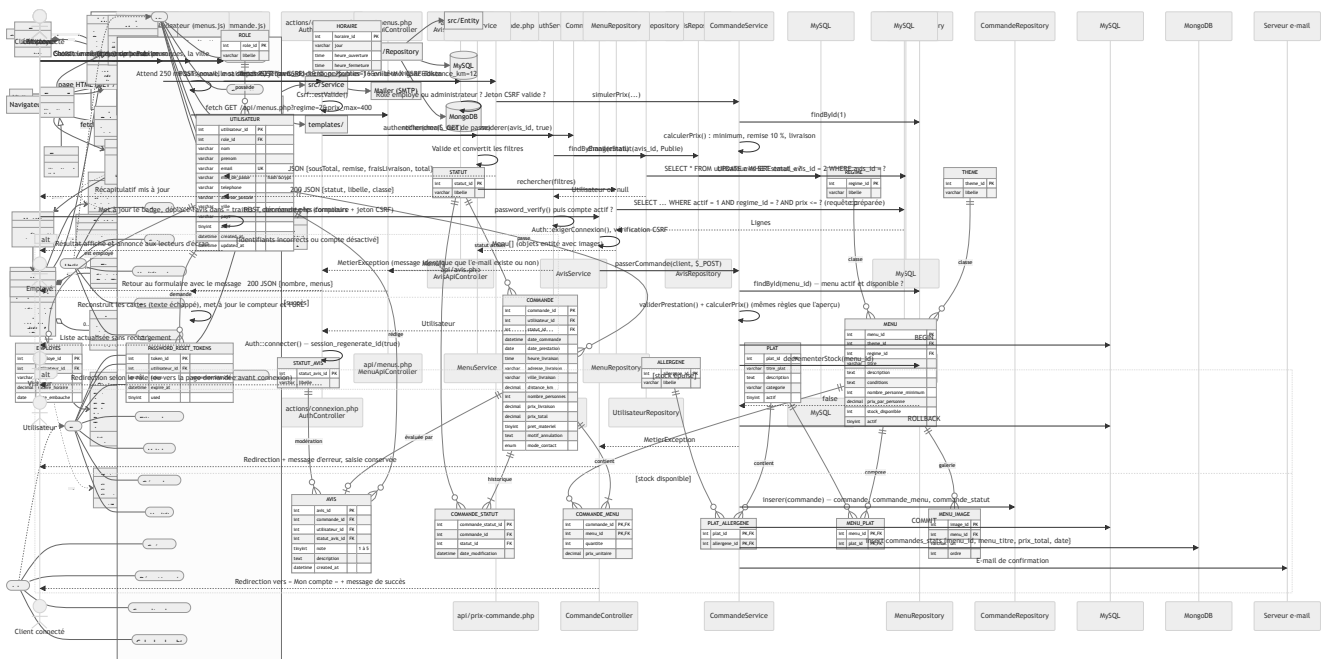
- présenter l'entreprise et les avis clients validés ;
- rendre les menus consultables par tous, avec des filtres qui actualisent la liste **sans rechargement** ;
- permettre aux clients de commander, suivre, modifier et noter leurs commandes ;
- donner aux employés un espace de gestion (commandes, menus, plats, horaires, avis) ;
- donner à l'administrateur les comptes employés et des statistiques issues d'une **base NoSQL** ;
- être **déployée, sécurisée et accessible (RGAA)**.

Les seules technologies imposées sont une base relationnelle et une base non relationnelle.

2. Choix de la stack

Besoin	Choix	Justification	Alternatives étudiées
Back-end	PHP 8.2 sans framework, architecture MVC en couches	Maîtriser les fondamentaux attendus par le titre (POO, PDO, sessions, sécurité) sans « magie » de framework ; hébergement simple (Apache)	Symfony / Laravel : plus productifs sur un gros projet, mais masquent les mécanismes que l'ECF doit démontrer
Accès aux données	PDO + repositories	Requêtes préparées natives (anti-injection), portabilité MySQL / MariaDB, retour d'objets entité	ORM (Doctrine) : surdimensionné pour une vingtaine de tables
Base relationnelle	MySQL 8	Données très structurées et liées (utilisateurs, menus, commandes) : clés étrangères, transactions, contraintes	PostgreSQL : équivalent ici ; MySQL est mieux supporté par phpMyAdmin et les hébergeurs visés
Base NoSQL	MongoDB 7	Statistiques de commandes : un document par commande, agrégé par menu et par période (<code>\$match</code> , <code>\$group</code>) sans jointure ; historique conservé même si un menu change	Redis : adapté au cache, pas aux agrégations ; tout en MySQL : ne répond pas à l'exigence NoSQL
Front-end	HTML 5, CSS (Bootstrap 5), JavaScript natif	Pages rendues par le serveur (référencement, accessibilité, fonctionnement sans JavaScript), enrichies par <code>fetch</code> là où l'interactivité compte	React / Vue : lourds pour quelques zones dynamiques ; jQuery / Axios : <code>fetch</code> est natif
Échanges asynchrones	API JSON (<code>api/*.php</code>) + <code>fetch</code>	Filtres des menus, prix de la commande, modération des avis et statistiques sans rechargement ; les règles restent côté serveur	Calculs en JavaScript seul : règles dupliquées et contournables
Graphiques	Chart.js	Léger, via CDN, graphiques en barres et camembert	D3.js : plus complexe pour ce besoin
E-mails	PHPMailer (SMTP)	Envoi fiable, configuration par variables d'environnement ; Mailpit en local	Fonction <code>mail()</code> : dépend de la configuration du serveur
Environnement	Docker / Docker Compose	Même environnement pour tous les développeurs et en production ; installation en une commande	WAMP / MAMP : versions différentes selon les postes
Hébergement	Railway	Déploiement direct du Dockerfile depuis GitHub, bases MySQL et MongoDB managées, HTTPS	Heroku : plus d'offre gratuite ; fly.io : volumes plus complexes ; Vercel : pas de PHP persistant
Gestion de projet	Notion (tableau Kanban)	Suivi des fonctionnalités par statut, documentation au même endroit	Trello, Jira
Maquettes	Figma	Wireframes et mockups ordinateur / mobile, partage de la charte graphique	Adobe XD

3. Architecture retenue



- **Un point d'entrée par page** (`pages/menus.php` ...) : les URL restent simples et lisibles, sans routeur.
- **Contrôleurs légers** : ils ne font que lire la requête, contrôler les droits et le jeton CSRF, puis déléguer.
- **Services** : toute la logique métier (voir le détail dans le diagramme de classes).
- **Repositories** : tout le SQL, qui renvoie des objets entité, jamais des tableaux bruts.
- **Templates** : uniquement de l'affichage ; aucune requête SQL.
- **Autoload PSR-4** (Composer) : chaque classe `App\...` est chargée automatiquement depuis `src/`.

Cette organisation permet de modifier une règle (ex. le tarif kilométrique) à un seul endroit (`CommandeService`), et de réutiliser le même code pour la page HTML et pour l'API JSON.

Configuration de l'environnement de travail

1. Outils

Outil	Usage
Visual Studio Code	Éditeur (extensions : PHP Intelephense, Docker, GitLens, Mermaid preview)
Docker Desktop	Exécution de l'application et des bases de données en conteneurs
Git + GitHub	Versionnement, dépôt public : https://github.com/Carlitaalt/vite-et-gourmand
Composer	Dépendances PHP (PHPMailer) et autoloader PSR-4 des classes
phpMyAdmin (conteneur)	Consultation de la base MySQL en local
Mailpit (conteneur)	Réception des e-mails envoyés en local, sans risque d'envoi réel
Figma	Wireframes, mockups et charte graphique
Notion	Gestion de projet (Kanban)

2. Conteneurs Docker (`docker-compose.yml`)

Service	Image	Port local	Rôle
app	php:8.2-apache + extensions pdo_mysql , mongodb	8080 (APP_PORT)	L'application
mysql	mysql:8.0	3306	Base relationnelle, initialisée par create.sql puis seed.sql
mongo	mongo:7	27017	Base NoSQL, initialisée par mongo- init.js
phpmyadmin	phpmyadmin	8081	Administration MySQL
mailpit	axllent/mailpit	8025	Boîte de réception de test

Le code source est monté dans le conteneur (`./:/var/www/html`) : une modification est visible immédiatement, sans reconstruire l'image.

3. Installation en local

```
git clone https://github.com/Carlitaalt/vite-et-gourmand.git
cd vite-et-gourmand
cp .env.example .env          # puis renseigner les mots de passe (valeurs libres en local)
composer install              # ou : docker compose run --rm app composer install
docker compose up -d --build
```

- Site : <http://localhost:8080> — phpMyAdmin : <http://localhost:8081> — e-mails : <http://localhost:8025>
- Remise à zéro complète des données : `docker compose down -v` puis `docker compose up -d`

4. Variables d'environnement

Aucun secret n'est écrit dans le code : tout passe par des variables d'environnement, lues avec `getenv()`. En local elles viennent du fichier `.env` (jamais versionné, voir `.gitignore`) ; en production, des variables du service Railway.

Variable	Exemple local	Rôle
<code>APP_PORT</code> , <code>APP_URL</code>	<code>8080</code> , <code>http://localhost:8080</code>	Port local, URL utilisée dans les liens des e-mails
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	<code>mysql</code> , <code>3306</code> , <code>vite_gourmand</code> ...	Connexion MySQL
<code>MYSQL_ROOT_PASSWORD</code>	—	Mot de passe root du conteneur MySQL (local uniquement)
<code>MONGO_URI</code> , <code>MONGO_DB</code>	<code>mongodb://mongo:27017</code> , <code>vite_gourmand</code>	Connexion MongoDB
<code>MAIL_HOST</code> , <code>MAIL_PORT</code> , <code>MAIL_USER</code> , <code>MAIL_PASSWORD</code> , <code>MAIL_ENCRYPTION</code>	<code>mailpit</code> , <code>1025</code> , vide, vide, <code>none</code>	Serveur SMTP
<code>MAIL_FROM</code> , <code>MAIL_CONTACT</code>	<code>noreply@...</code> , <code>contact@...</code>	Expéditeur et destinataire des demandes de contact

5. Organisation Git

- `main` : version stable, déployée automatiquement.
- `dev` : branche d'intégration.
- `feature/...` : une branche par fonctionnalité, créée depuis `dev` , fusionnée dans `dev` après test, puis `dev` fusionnée dans `main` .

Exemple : la refactorisation en couches et les appels `fetch` ont été développés sur `feature/architecture-poo` , en commits thématiques (environnement, couches métier, contrôleurs et templates, API fetch).

6. Structure du projet

```
vite-et-gourmand/  
├─ pages/           points d'entrée des pages (3 lignes : appel du contrôleur)  
├─ actions/         points d'entrée des formulaires de connexion / inscription /  
déconnexion  
├─ api/             points d'entrée JSON appelés en fetch  
├─ src/  
│   ├─ Controller/  contrôleurs (et Controller/Api pour le JSON)  
│   ├─ Service/     logique métier  
│   ├─ Repository/  accès aux données MySQL et MongoDB  
│   ├─ Entity/      objets métier  
│   ├─ Enum/        rôles et statuts  
│   ├─ Core/        connexion BDD, vues, e-mails, messages flash  
│   ├─ Security/    session, rôles, jeton CSRF  
│   └─ bootstrap.php  
├─ templates/       vues HTML (layout, pages, onglets partagés admin/employé)  
├─ assets/           CSS, JavaScript, images  
├─ database/         create.sql, seed.sql, mongo-init.js  
├─ docs/             documentation technique et manuel  
├─ Dockerfile, docker-compose.yml, docker-entrypoint.sh  
└─ .env.example
```

Modèle conceptuel de données — Vite & Gourmand

Le modèle a été conçu à partir du cahier des charges, avant le développement. Il est implémenté dans MySQL par `database/create.sql` (structure) et `database/seed.sql` (données de référence et de démonstration).

1. Entités et associations (notation Merise)

Association	Entité A	Card. A	Entité B	Card. B	Données portées
possède	UTILISATEUR	1,1	ROLE	0,n	
est employé	EMPLOYE	1,1	UTILISATEUR	0,1	
demande (réinitialisation)	JETON_REINITIALISATION	1,1	UTILISATEUR	0,n	
passé	COMMANDE	1,1	UTILISATEUR	0,n	
contient	COMMANDE	1,n	MENU	0,n	quantité, prix unitaire
est dans l'état (actuel)	COMMANDE	1,1	STATUT	0,n	
a traversé (historique)	COMMANDE	1,n	STATUT	0,n	date de modification
appartient à	MENU	1,1	THEME	0,n	
respecte	MENU	1,1	REGIME	0,n	
illustre	IMAGE	1,1	MENU	0,n	
compose	MENU	0,n	PLAT	0,n	
contient (allergène)	PLAT	0,n	ALLERGENE	0,n	
évalue	AVIS	1,1	COMMANDE	0,1	
rédige	AVIS	1,1	UTILISATEUR	0,n	
est modéré	AVIS	1,1	STATUT_AVIS	0,n	

`HORAIRE` est une entité indépendante (un enregistrement par jour de la semaine).

Choix de conception :

- **Un plat peut appartenir à plusieurs menus** (association n,n `compose`), comme le demande le cahier des charges.
- **Le statut actuel est stocké dans la commande**, et chaque changement est aussi historisé dans `commande_statut` : on affiche rapidement le statut courant, et le client dispose du suivi complet (statut + date et heure).

- **Le prix unitaire est copié dans** `commande_menu` au moment de la commande : si le prix du menu change ensuite, les commandes passées gardent leur prix réel.
- **Les rôles, statuts, thèmes et régimes sont des tables de référence** : on peut en ajouter sans modifier le code (le cahier des charges permet d'enrichir les régimes).
- **Les jetons de réinitialisation ne sont stockés que sous forme d'empreinte SHA-256.**

2. Modèle physique (MySQL)

3. Intégrité des données

- **Clés étrangères** sur toutes les relations (moteur InnoDB).
- **ON DELETE CASCADE** pour les données qui n'ont pas de sens seules : images et liaisons d'un menu, allergènes d'un plat, historique d'une commande, jetons d'un utilisateur.
- **Pas de cascade** entre commande, utilisateur et menu : on ne peut pas supprimer un menu déjà commandé (il faut le désactiver), et un compte client supprimé est anonymisé pour conserver les commandes (obligations comptables).
- Contrainte **CHECK** (`note BETWEEN 1 AND 5`) sur les avis, **UNIQUE** sur l'e-mail des utilisateurs.

4. Données non relationnelles (MongoDB)

Base `vite_gourmand`, collection `commandes_stats`. Un document est ajouté à chaque commande :

```
{
  "commande_id": 12,
  "menu_id": 3,
  "menu_titre": "Menu de Noël",
  "prix_total": 550.0,
  "date": { "$date": "2026-09-12T09:30:00Z" }
}
```

L'espace administrateur les agrège (`$match` sur la période, puis `$group` par menu) pour afficher le nombre de commandes par menu dans un graphique. Ce modèle « journal d'événements » s'agrège simplement sans jointure, et l'historique statistique reste intact même si un menu est modifié ou supprimé dans MySQL.

Diagramme de classes — architecture en couches

L'application est découpée en couches qui ont chacune une seule responsabilité :

Couche	Dossier	Rôle
Point d'entrée	pages/ , actions/ , api/	Charge <code>src/bootstrap.php</code> puis appelle un contrôleur (3 lignes)
Contrôleur	src/Controller	Lit la requête, vérifie les droits et le jeton CSRF, appelle un service, choisit la réponse (page, redirection, JSON)
Service	src/Service	Logique métier : calcul du prix, règles de commande, e-mails, statistiques
Repository	src/Repository	Accès aux données (SQL préparé, MongoDB), transforme les lignes en objets entité
Entité	src/Entity , src/Enum	Objets métier et leurs règles (ex : une commande refuse une transition de statut interdite)
Vue	templates/	HTML uniquement, alimenté par les données préparées par le contrôleur

Une dépendance ne va jamais « vers le haut » : un repository ne connaît pas les services, une entité ne connaît pas la base de données.

1. Modèle métier (entités)

2. Couches : exemple de la commande

Principes de programmation orientée objet appliqués :

- **Encapsulation** : propriétés privées, modifiées uniquement par des méthodes métier qui contrôlent les règles (`Commande::passerAuStatut()` lève une `MetierException` si la transition est interdite).
- **Héritage** : `AbstractController`, `AbstractRepository`, `Reference` (Theme, Regime, Allergene) ; l'espace administrateur hérite de l'espace employé et n'ajoute que ses actions propres.
- **Polymorphisme** : `EspaceAdminController::traiterAction()` redéfinit la méthode parente et délègue à `parent::traiterAction()` pour les actions communes.
- **Injection de dépendances** : chaque service reçoit ses repositories dans son constructeur (valeurs par défaut), ce qui permet de les remplacer pour les tests.
- **Enums PHP 8.1** : les statuts et rôles sont des types, pas des nombres « magiques » dispersés dans le code.

Diagramme de cas d'utilisation

Quatre acteurs, avec héritage des droits : un **utilisateur** connecté peut faire tout ce que fait un **visiteur** ; l'**administrateur** peut faire tout ce que fait un **employé**.

Règles associées

Cas d'utilisation	Règles de gestion
Créer un compte	Nom, prénom, GSM, e-mail, adresse postale obligatoires ; mot de passe ≥ 10 caractères avec majuscule, minuscule, chiffre et caractère spécial ; rôle « utilisateur » ; e-mail de bienvenue
Commander un menu	Connexion obligatoire (redirection vers la connexion puis retour à la commande) ; minimum de personnes du menu ; -10% dès 5 personnes au-delà du minimum ; livraison gratuite à Bordeaux sinon $5\text{ €} + 0,59\text{ €/km}$; stock décrémenté ; e-mail de confirmation
Modifier / annuler	Possible tant que la commande est « en attente » ; tout est modifiable sauf le menu ; une annulation libère le stock
Donner un avis	Commande terminée, une seule fois ; note de 1 à 5 et commentaire ; visible sur l'accueil après validation
Traiter les commandes	Transitions imposées : en attente $\rightarrow$ acceptée $\rightarrow$ en préparation $\rightarrow$ en cours de livraison $\rightarrow$ livrée $\rightarrow$ (retour de matériel si prêt) $\rightarrow$ terminée ; e-mails automatiques au retour de matériel (600 € après 10 jours ouvrés) et à la fin
Annuler (employé)	Mode de contact (GSM ou e-mail) et motif obligatoires ; client prévenu par e-mail
Créer un compte employé	Réservé à l'administrateur ; l'e-mail envoyé ne contient pas le mot de passe ; impossible de créer un administrateur

Diagrammes de séquence

1. Filtrer les menus (appel asynchrone fetch)

2. Passer une commande

3. Connexion

4. Modérer un avis (appel asynchrone fetch)

Mesures et bonnes pratiques de sécurité

Chaque mesure est présentée avec la menace qu'elle traite, sa mise en œuvre dans le code et sa justification. Les menaces sont rattachées au **Top 10 OWASP 2021**, référence utilisée pour la veille.

1. Tableau de synthèse

#	Menace (OWASP)	Mesure	Où dans le code
1	Injection SQL (A03)	Requêtes préparées PDO, sans émulation	<code>src/Repository/*</code> , <code>Core/Database.php</code>
2	Cross-Site Scripting (A03)	Échappement à l'affichage (serveur et navigateur)	<code>templates/*</code> , <code>main.js: echapperHtml()</code>
3	Falsification de requête (CSRF) (A01)	Jeton secret par session sur chaque POST et chaque fetch	<code>Security/Csrf.php</code> , <code>AbstractController::verifierCsrf()</code>
4	Contrôle d'accès défaillant (A01)	Rôles hiérarchiques + vérification de propriété des données	<code>Security/Auth.php</code> , <code>services</code>
5	Stockage des mots de passe (A02)	Hachage bcrypt (<code>password_hash</code>)	<code>AuthService</code> , <code>EmployeService</code>
6	Authentification (A07)	Politique de mot de passe, messages neutres, session sécurisée	<code>AuthService</code> , <code>bootstrap.php</code>
7	Réinitialisation de mot de passe (A07)	Jeton aléatoire, à usage unique, 1 h, stocké haché	<code>TokenReinitialisationRepository</code>
8	Téléversement de fichiers (A04)	Type réel, liste blanche, taille, nom généré	<code>MenuService::ajouterImages()</code>
9	Secrets exposés (A05)	Variables d'environnement, <code>.env</code> exclu de Git et de l'image	<code>.gitignore</code> , <code>.dockerignore</code> , <code>Core/*</code>
10	Mauvaise configuration (A05)	Erreurs masquées en production, journalisées	<code>Dockerfile</code> , <code>bootstrap.php</code>
11	Redirection ouverte (A01)	Redirection après connexion limitée aux chemins internes	<code>AuthController::connexion()</code>
12	Intégrité des données (A04)	Transactions, clés étrangères, contraintes	<code>Database::transaction()</code> , <code>create.sql</code>

2. Détail des mesures

2.1 Injections SQL

- **Toutes** les requêtes utilisent `prepare()` / `execute()` avec des marqueurs `?` ; aucune valeur saisie n'est concaténée dans du SQL.
- `PDO::ATTR_EMULATE_PREPARES => false` : la requête et les données sont envoyées séparément à MySQL (vraies requêtes préparées).
- Les filtres dynamiques (menus, chiffre d'affaires) ajoutent des **fragments SQL constants** et des paramètres, jamais du texte utilisateur.
- Les identifiants reçus sont convertis en entiers (`(int)`) et les listes (thèmes, statuts) sont validées côté serveur.

2.2 Cross-Site Scripting (XSS)

- Côté serveur : toute donnée affichée passe par `htmlspecialchars()` dans les templates, y compris dans les e-mails HTML.
- Côté navigateur : le JavaScript insère le texte reçu de l'API via `textContent` ou `echapperHtml()`.
- Les données JSON intégrées dans une page (statistiques) sont encodées avec `JSON_HEX_TAG` | `JSON_HEX_AMP` : un titre contenant `</script>` ne peut pas s'échapper de la balise.

2.3 Falsification de requête inter-sites (CSRF)

- Un jeton de 256 bits (`random_bytes(32)`) est généré par session.
- Il est ajouté en champ caché à **tous** les formulaires POST (`Csrf::champ()`) et envoyé en en-tête `X-CSRF-Token` par `appelApi()` pour les appels fetch qui modifient des données.
- Il est comparé avec `hash_equals()` (comparaison en temps constant). Un jeton absent ou faux entraîne un refus (message et redirection, ou HTTP 400 pour l'API).
- Le cookie de session est en `SameSite=Lax`, une seconde barrière contre les requêtes venant d'autres sites.

2.4 Contrôle d'accès

- Chaque page protégée appelle `Auth::exigerConnexion()` ou `Auth::exigerRole()` **avant tout traitement** ; l'API renvoie HTTP 403.
- Hiérarchie des rôles (`Role::aAuMoins()`) : l'administrateur hérite des droits de l'employé.
- **Vérification de propriété** : un client ne peut modifier, annuler ou noter que ses propres commandes (`CommandeService::commandeDuClient()`, `AvisService::deposer()`). Changer un identifiant dans le formulaire ne donne pas accès à la commande d'un autre client (protection IDOR).
- La désactivation de compte vérifie qu'il s'agit bien d'un employé : impossible de désactiver l'administrateur.
- Il n'existe aucun formulaire de création de compte administrateur (exigence du client).

2.5 Mots de passe

- Hachage avec `password_hash(PASSWORD_DEFAULT)` (bcrypt, sel aléatoire intégré, coût adaptatif) et vérification avec `password_verify()`.
- Politique imposée **côté serveur** (`AuthService::REGEX_MOT_DE_PASSE`) : 10 caractères minimum dont une majuscule, une minuscule, un chiffre et un caractère spécial. Le navigateur affiche les

règles en direct, mais seul le serveur fait foi.

- Le mot de passe d'un nouvel employé n'est jamais envoyé par e-mail.

2.6 Authentification et session

- Même message « Email ou mot de passe incorrect » que le compte existe ou non (pas d'énumération des comptes).
- `session_regenerate_id(true)` à la connexion : protection contre la fixation de session.
- Cookie de session `HttpOnly` (inaccessible au JavaScript, donc à un éventuel XSS), `Secure` en HTTPS, `SameSite=Lax`, et `use_strict_mode` (refus des identifiants de session inventés).
- Déconnexion : session vidée, cookie supprimé, session détruite.

2.7 Réinitialisation du mot de passe

- Jeton de 64 caractères hexadécimaux (`random_bytes(32)`), valable 1 heure, à usage unique, un seul jeton actif par compte.
- **Seule l'empreinte SHA-256 est stockée** : une fuite de la base ne permet pas d'utiliser les liens envoyés.
- Réponse identique que l'adresse existe ou non.

2.8 Téléversement des photos de menu

- Le type est déterminé à partir du **contenu réel** du fichier (`info`), et non de son extension ni du type annoncé par le navigateur.
- Liste blanche : JPEG, PNG, WEBP ; 5 Mo maximum par fichier.
- Le **nom du fichier est généré par le serveur** (`menu_<id>_<aléatoire>.<extension>`) : impossible de déposer un `.php` ou d'écraser un fichier existant.
- Seuls les fichiers du dossier `assets/images/menus/` peuvent être supprimés par l'application.

2.9 Gestion des secrets

- Identifiants MySQL, MongoDB et SMTP lus depuis des variables d'environnement.
- `.env` exclu de Git (`.gitignore`) et de l'image Docker (`.dockerignore`) ; un `.env.example` sans secret documente les variables.
- Veille / incident : des identifiants avaient été versionnés dans un fichier suivi par Git avant d'être ignorés. Correction : rotation des mots de passe, retrait du suivi (`git rm --cached`), passage aux variables d'environnement. Même traitement pour des identifiants SMTP de test trouvés dans le code lors de la refactorisation.

2.10 Erreurs et journalisation

- En production : `display_errors = Off`, erreurs journalisées sur la sortie d'erreur (consultables dans les logs Railway).
- Un gestionnaire global d'exceptions (`bootstrap.php`) journalise l'erreur technique et n'affiche qu'un message générique.
- Les erreurs **métier** (`MetierException` : minimum de personnes non atteint, statut interdit...) ont un message pensé pour l'utilisateur ; les détails SQL ne sont jamais affichés.

2.11 Intégrité des données

- Transactions (`Database::transaction()`) pour les opérations en plusieurs étapes : commande + décrémentation du stock, annulation + remise en stock, création d'un employé (compte + fiche).
- Clés étrangères, contraintes `UNIQUE` (e-mail) et `CHECK` (note de 1 à 5) dans `create.sql`.
- Le prix est **toujours recalculé par le serveur** au moment de l'enregistrement : le prix affiché dans le navigateur n'est qu'un aperçu.

3. RGPD

- Données collectées limitées au besoin (identité, contact, adresse de livraison).
- Droit d'accès et de rectification : page « Mon profil ». Droit à l'effacement : suppression du compte, ou anonymisation s'il existe des commandes à conserver pour la comptabilité.
- Un seul cookie, strictement nécessaire (session) : pas de bandeau de consentement requis.
- Mentions légales détaillant l'usage des données.

4. Limites connues et améliorations prévues

- Limitation du nombre de tentatives de connexion (anti force brute), par exemple par adresse IP.
- En-têtes HTTP de sécurité (`Content-Security-Policy` , `X-Frame-Options`) à ajouter dans la configuration Apache.
- Analyse automatique des dépendances (`composer audit`) dans une intégration continue.

Documentation du déploiement

L'application est déployée sur **Railway** à partir du `Dockerfile` du dépôt GitHub. Cette procédure permet à un tiers de reproduire la mise en ligne de zéro.

1. Architecture de production

2. Préparation de l'image (`Dockerfile`)

1. Image officielle `php:8.2-apache` , extensions `pdo_mysql` (MySQL) et `mongodb` (installée par PECL).
2. Un seul module MPM Apache activé (`mpm_prefork` , requis par le module PHP) — voir « Problèmes rencontrés ».
3. `display_errors = Off` , erreurs envoyées sur la sortie d'erreur (lisibles dans les logs Railway).
4. Limites d'upload augmentées (photos des menus).
5. `composer install --no-dev --optimize-autoloader` : dépendances et autoloader des classes.
6. `docker-entrypoint.sh` : Railway impose le port d'écoute via la variable `$PORT` ; le script l'applique à Apache au démarrage.
7. `.dockerignore` : le fichier `.env` , le dossier `.git` et la documentation ne sont pas copiés dans l'image.

3. Étapes de mise en ligne

Étape 1 — Créer le projet et les bases de données

1. Se connecter sur <https://railway.com> avec son compte GitHub.
2. **New Project** → **Deploy from GitHub repo** → choisir `Carlitaalt/vite-et-gourmand` . Railway détecte le `Dockerfile` .
3. Dans le projet : + **Create** → **Database** → **MySQL**, puis + **Create** → **Database** → **MongoDB**.

Étape 2 — Variables d'environnement du service app

Onglet **Variables** du service de l'application (les références `{{...}}` sont résolues par Railway) :

Variable	Valeur
DB_HOST	<code>\${MySQL.MYSQLHOST}</code>
DB_PORT	<code>\${MySQL.MYSQLPORT}</code>
DB_NAME	<code>\${MySQL.MYSQLDATABASE}</code>
DB_USER	<code>\${MySQL.MYSQLUSER}</code>
DB_PASSWORD	<code>\${MySQL.MYSQLPASSWORD}</code>
MONGO_URI	<code>\${MongoDB.MONGO_URL}</code>
MONGO_DB	<code>vite_gourmand</code>
APP_URL	URL publique de l'application (étape 4)
MAIL_HOST, MAIL_PORT, MAIL_USER, MAIL_PASSWORD, MAIL_ENCRYPTION, MAIL_FROM, MAIL_CONTACT	Identifiants du fournisseur d'e-mails

Aucun de ces secrets n'apparaît dans le dépôt.

Étape 3 — Initialiser les bases de données

Avec les informations de connexion publiques du service MySQL (onglet **Connect**) :

```
mysql -h <hôte public> -P <port public> -u root -p <base> < database/create.sql
mysql -h <hôte public> -P <port public> -u root -p <base> < database/seed.sql
mongosh "<MONGO_PUBLIC_URL>" database/mongo-init.js
```

Étape 4 — Rendre l'application accessible

Service app → **Settings** → **Networking** → **Generate Domain** : Railway fournit une URL `https://...up.railway.app` avec certificat HTTPS. Reporter cette URL dans la variable `APP_URL`.

Étape 5 — Vérifier

- Onglet **Deployments** : le déploiement doit être « Active » ; **View logs** affiche les erreurs PHP éventuelles.
- Parcours de contrôle : accueil, filtres des menus, connexion avec les comptes de démonstration, commande, espace employé, statistiques administrateur.

Déploiement continu

Chaque `git push` sur `main` déclenche une reconstruction et un redéploiement automatiques. Le travail se fait sur `dev` et les branches `feature/...` ; seul un code testé est fusionné dans `main`.

4. Problèmes rencontrés et solutions

Problème	Cause	Solution
AH00534: More than one MPM loaded au démarrage	L'image charge plusieurs modules MPM d'Apache	Suppression de tous les <code>mpm_*</code> puis activation de <code>mpm_prefork</code> seul (Dockerfile + <code>docker-entrypoint.sh</code>)
Application inaccessible	Railway attribue un port dynamique (<code>\$PORT</code>)	Réécriture de <code>Listen</code> et du VirtualHost au démarrage (<code>docker-entrypoint.sh</code>)
Erreurs PHP visibles des visiteurs	Configuration par défaut de l'image	<code>display_errors = Off</code> + journalisation sur la sortie d'erreur
Photos téléversées perdues après un redéploiement	Le système de fichiers d'un conteneur est éphémère	Volume Railway monté sur <code>/var/www/html/assets/images/menus</code>